



ITI · INTRODUCTION TO PYTHON TRACK

Python Programming for AI Applications

Eng. Fatma Elraey*Day 3 — Modules, Exceptions, Files & the AI Library Ecosystem*

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Day 3 of 6
Focus	Organizing code into modules, handling errors safely, reading/writing data files, and touring the libraries every AI project relies on

Quick Recap — Day 2

On Day 2 you moved from procedural scripts to functions — writing reusable, parameterized blocks of logic, understanding scope, and returning values instead of just printing them. Today we go one level up: organizing those functions into modules and packages, protecting your programs from bad data with exception handling, and reading/writing the files that every dataset, model, and log ultimately lives in.

Day 3 Outline

1. Modules & Packages for AI Projects
2. Exception Handling in ML Pipelines
3. Reading & Writing Data Files
4. Standard Libraries (os, math, re)
5. External Libraries (numpy, pandas, scikit-learn, PyTorch...)
6. Day Tips — Pythonic Tricks

01

Modules & Packages for AI Projects

Every real AI project — from a weekend Kaggle notebook to a production ML pipeline — is organized into modules: separate .py files that each own one responsibility. A module is nothing more than a Python file whose functions and variables you can import wherever you need them.

From One Script to a Real Project

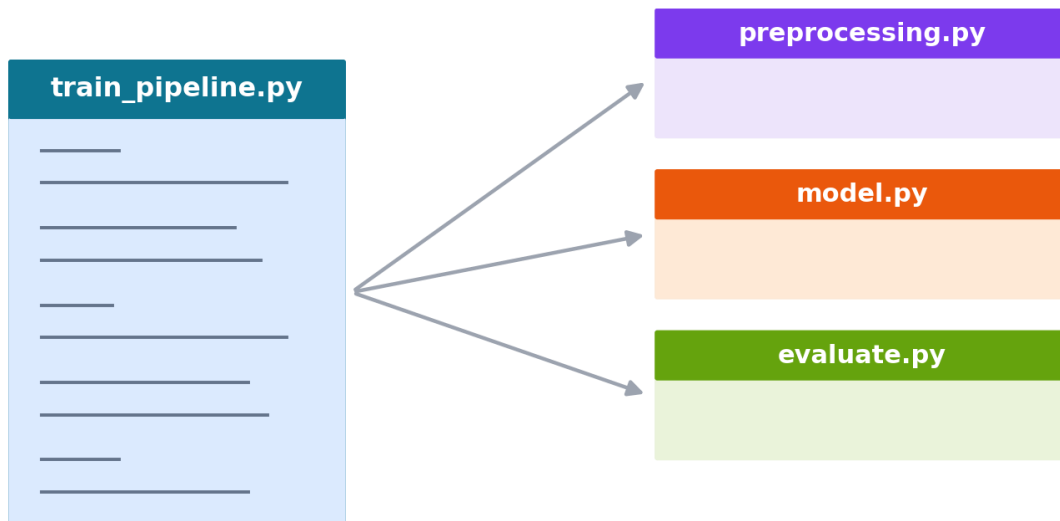


Figure 1.1 — A training pipeline importing preprocessing, model, and evaluation modules.

You bring a module's contents into your file with `import`, or pull out just what you need:

From **module_name** **import** **block_name**

```
# math.py provides mathematical functions
from math import sqrt

rmse = sqrt(mean_squared_error)
```

A package is a folder of related modules grouped under one name — exactly how libraries like `scikit-learn` or `numpy` are organized internally, and how you'll structure your own AI projects once they outgrow a single file.

A Package: ai_toolkit/

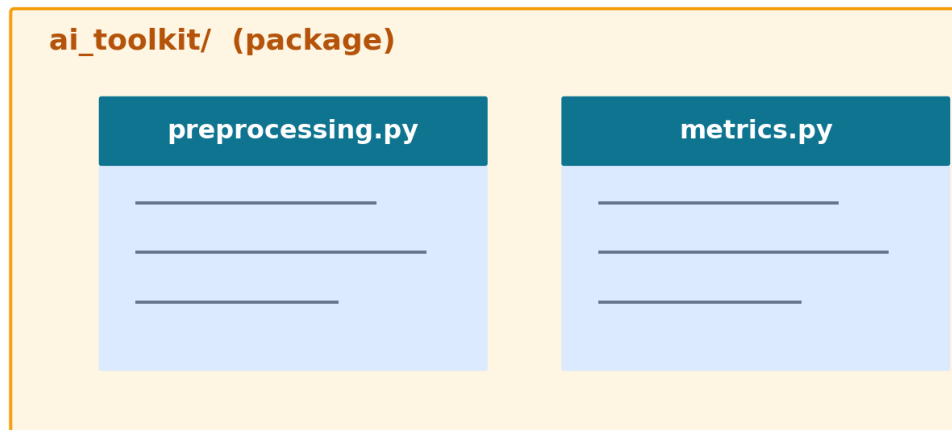


Figure 1.2 — Two modules, `preprocessing.py` and `metrics.py`, grouped inside one package.

From `package_name.module_name` import `block_name`

```

from ai_toolkit.preprocessing import normalize
from ai_toolkit.metrics import accuracy

X = normalize(raw_features)
score = accuracy(y_true, y_pred)

```

AI CONNECTION *scikit-learn, PyTorch, and TensorFlow are all just packages — folders of modules you import from every day. Once you write `sklearn.preprocessing.StandardScaler()` or `torch.nn.Linear()`, you're using exactly this pattern.*

✓ QUICK CHECK — Modules & Packages

Q1. What single line lets you import only the `sqrt` function out of the `math` module?

Answer: `from math import sqrt` — this pulls in just that name instead of the whole module.

Q2. What is a package, structurally?

Answer: A directory containing multiple related module (`.py`) files, importable as one grouped namespace, e.g. `ai_toolkit.preprocessing`.

Q3. Why split an AI project into modules instead of one big script?

Answer: Each file stays focused on one job (loading data, building the model, evaluating it), which makes the codebase easier to test, reuse, and hand off to teammates.

02

Exception Handling in ML Pipelines

Errors in Python fall into three families. Syntax errors stop your code from even running — Python catches these before executing a single line. Logical errors run fine but produce the wrong result (a bug you have to reason through). The third family, exceptions, are runtime errors — problems that only surface while the program is executing, like a missing file or a division by zero in the middle of a training loop.

Three Kinds of Errors



Figure 2.1 — Syntax errors block execution; logical errors give a wrong-but-silent result; exceptions are runtime failures you can catch.

A syntax error — Python refuses to even start:

```
print("Missing the closing quote")  
# SyntaxError: unterminated string literal
```

An exception — the code is valid, but something goes wrong while it's running:

```
accuracy = correct / total  
# ZeroDivisionError: division by zero (if total == 0)
```

The `try / except / else / finally` block lets you catch an exception instead of letting it crash your whole pipeline — essential when you're looping over thousands of rows in a dataset and one bad row shouldn't kill the run.

The `try / except / else / finally` Flow

try	Run the risky code (e.g. parsing a data row)
except	Handle the exception (e.g. log a bad row, skip it)
else	Runs only if no exception (e.g. add row to clean dataset)
finally	Always runs (e.g. close the file handle)

Figure 2.2 — *try* runs the risky code, *except* catches failures, *else* runs only on success, *finally* always runs.

Without exception handling:

```
numbers = ["10", "5", "0", "oops", "2"]

for n in numbers:
    result = 100 / int(n)
    print(result)
```

With exception handling:

```
numbers = ["10", "5", "0", "oops", "2"]

for n in numbers:
    try:
        result = 100 / int(n)
    except ValueError:
        print(n, "-> not a number")
    except ZeroDivisionError:
        print(n, "-> can't divide by zero")
    else:
        print(n, "-> 100 /", n, "=", result)
    finally:
        print("  checked", n)
```

You can also raise your own exceptions to enforce assumptions about your data, exactly like a validation step before training:

```
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

ages = [25, 10, -5, 30]

for a in ages:
    try:
        valid_age = check_age(a)
    except ValueError as e:
        print(a, "-> rejected:", e)
    else:
        print(a, "-> accepted")
```

If we want to take inputs from the user using input function:

```
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

while True:
    text = input("Enter an age (or 'quit' to stop): ")

    if text == "quit":
        break

    try:
        age = int(text)
        valid_age = check_age(age)
    except ValueError as e:
        print("  Rejected:", e)
    else:
        print("  Accepted:", valid_age)
```

Most Used Exceptions: AssertionError · TypeError · IndexError · NameError · ValueError · ZeroDivisionError



AI CONNECTION *Production ML pipelines wrap almost every data-loading and preprocessing step in try/except — a single malformed row, missing label, or NaN value should never be allowed to crash a training run processing millions of examples.*

✓ QUICK CHECK — Exception Handling

Q1. What's the difference between a syntax error and an exception?

Answer: A syntax error stops the program before it runs at all; an exception happens while the program is executing valid code.

Q2. Which block runs only if the try block succeeds with no exception?

Answer: The else block — it runs only when nothing in try raised an exception.

Q3. Which block always executes, whether or not an exception occurred?

Answer: finally — commonly used to close files or release resources no matter what happened.

03

Reading & Writing Data Files

Every dataset you'll ever train on starts life as a file on disk. Python opens files with `open(file_name, mode)` — the mode controls whether you're reading, writing, or appending, and whether the data is text or binary.

File Modes

r	w	a	r+
Read only	Write (overwrite / create)	Append (create if missing)	Read & write

Figure 3.1 — *r* reads, *w* overwrites (or creates), *a* appends, *r+* does both.

Reading a file — the whole thing, a fixed number of characters, one line, or line-by-line in a loop:

```
f1 = open("predictions.csv", "r")
print(f1.read()) # entire file as one string
f1.close()
f1 = open("predictions.csv", "r")
print(f1.read(4)) # first 4 characters only

print(f1.readline())# just the next line

f1 = open("predictions.csv", "r")
for line in f1:
    print(line)      # iterate row by row – the standard way to stream a dataset
```

Writing and appending — note that mode "w" overwrites the whole file, while "a" adds on to the end:

```
f1 = open("run_log.txt", "w") # create a file if not exist
f1.write("epoch=1, loss=0.842")
f1.close()

f1 = open("run_log.txt", "a")
f1.write("\nepoch=2, loss=0.617") # appended, not overwritten
f1.close()
```

The with statement handles opening and closing automatically — this is the pattern you should use in real code:

```
with open("dataset.txt", "r") as fp:
    lines = fp.readlines()
    print(lines)
# file is automatically closed here, even if an error occurs
```

Files content to use:

predictions.csv

```
image_id,predicted_label,confidence
img_001,cat,0.94
img_002,dog,0.87
```




```
img_003,cat,0.76  
img_004,bird,0.99  
img_005,dog,0.61
```

dataset.txt

```
4.5,0.2,setosa  
3.1,0.4,versicolor  
5.0,0.9, virginica  
2.2,0.1,setosa
```

AI CONNECTION *pandas.read_csv(), torch.load(), and every dataset loader you'll use are built on exactly this open/read pattern under the hood — understanding file I/O demystifies what those higher-level functions are actually doing.*

✓ QUICK CHECK — File I/O

Q1. What does mode "a" do differently from mode "w"?

Answer: "a" appends new content to the end of the existing file; "w" erases the file's previous content before writing.

Q2. What is the most Pythonic way to read a file, and why?

Answer: Using with open(...) as f: — it automatically closes the file for you, even if an exception is raised inside the block.

Q3. How do you read a dataset file one row at a time without loading it all into memory?

Answer: for line in file: — this iterates line by line, which is memory-efficient for large datasets.

04

Standard Libraries

Python ships with a huge "standard library" — modules that are always available, no installation required. Three you'll reach for constantly in AI work: `os` for file-system paths, `math` for numerical functions, and `re` for text pattern matching.

`os` — talking to the operating system

```
import os

os.getcwd()           # current working directory
os.listdir("datasets/") # list files in a dataset folder
os.path.join("datasets", "train.csv") # build a path safely across OSes
```

`math` — numerical functions

```
import math

math.sqrt(loss)           # e.g. converting MSE to RMSE
math.ceil(batches / 2)    # round up when splitting a dataset
math.floor(3.9)           # 3
math.pi                   # 3.14159...
```

`re` — regular expressions

```
import re

re.match(pattern, string) # match starting at the beginning of the string
re.fullmatch(pattern, string) # match the entire string against the pattern
re.search(pattern, string) # scan the whole string for the pattern
```

AI CONNECTION `re` is exactly what powers text preprocessing before feeding data into an NLP model — stripping punctuation, extracting hashtags, or validating input formats before tokenization.

✓ QUICK CHECK — Standard Libraries

Q1. Which module would you use to build a file path that works on both Windows and Linux?

Answer: `os` — specifically `os.path.join()`, which handles the correct path separator for the current OS.

Q2. Which function converts a mean-squared-error value into root-mean-squared-error?

Answer: `math.sqrt()` — RMSE is just the square root of MSE.

Q3. Which module would you reach for to strip out unwanted characters from raw text before tokenizing it?

Answer: `re` — regular expressions are the standard tool for pattern-based text cleaning.

05

External Libraries

Beyond the standard library, Python's real power for AI comes from external libraries — code other people wrote and published, which you install with `pip`, Python's package manager.

Standard Library vs. External Libraries



`pip install <library-name>`

Figure 5.1 — Standard library modules ship with Python; external libraries are installed on demand with `pip`.

```

pip install numpy

import numpy as np
arr = np.array([1, 2, 3])
arr.mean()
  
```

The libraries you'll meet across this track and beyond:

Library	What it's for
<code>numpy</code>	Fast array math — the numerical foundation almost every other AI library sits on top of

Library	What it's for
pandas	Opening files and building DataFrames — the standard way to load and explore tabular datasets
matplotlib	Plotting — visualizing training curves, distributions, and model results
scikit-learn	Classical ML algorithms — regression, classification, clustering, and evaluation metrics
PyTorch / TensorFlow	Deep learning — building and training neural networks
Django / Flask / CherryPy	Web frameworks — serving a trained model behind an API
Arcade / PyGame	Game development libraries — useful for building reinforcement-learning environments

AI CONNECTION *This is the exact stack of an AI project: pandas to load the data, numpy underneath it doing the math, scikit-learn or PyTorch to build the model, matplotlib to visualize results, and Flask to serve the trained model to the world.*

✓ QUICK CHECK — External Libraries

Q1. What command installs an external library like numpy?

Answer: `pip install numpy` — `pip` is Python's package manager for third-party libraries.

Q2. Which library would you use to load a CSV dataset into a DataFrame?

Answer: `pandas` — it's built specifically for opening files and working with tabular data.

Q3. Name two libraries used for building and training neural networks.

Answer: `PyTorch` and `TensorFlow` are the two dominant deep-learning libraries.

06

Day Tips — Pythonic Tricks

Tip 1 — Sequence Unpacking

```
scores = [0.91, 0.87, 0.94, 0.88]
a, b, c, d = scores
# a=0.91, b=0.87, c=0.94, d=0.88

first, *rest = scores
# first=0.91, rest=[0.87, 0.94, 0.88]
```

Tip 2 — with

```
with open("model_metrics.txt", "r") as fp:
    fp.read()
# handles setup (open) and teardown (close) automatically
```

Tip 3 — List Comprehensions

```
# [ output for item in sequence if condition ]
squared_evens = [x**2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]

normalized = [(x - min_val) / (max_val - min_val) for x in raw_features]
```

Tip 4 — lambda Functions

```
# Normal function
def double(x):
    return x + x

# Equivalent Lambda
double = lambda x: x + x
```

AI CONNECTION You'll write `df['col'].apply(lambda x: x.strip())` in almost every pandas preprocessing script — lambdas are the go-to tool for quick, throwaway transforms applied across a dataset.

✓ QUICK CHECK — Day Tips

Q1. What does `a, *b, c = [1, 13, 3, 7]` assign to `b`?

Answer: `b = [13, 3]` — the `*` collects all the 'middle' elements into a list.

Q2. What are the three parts of a list comprehension?

Answer: An output expression, an input sequence (for ... in ...), and an optional condition (if ...).

Q3. When would you reach for a lambda instead of `def`?

Answer: For small, one-line, throwaway functions — especially as an argument to something like `sorted()`, `map()`, or `DataFrame.apply()`.

07

Practical Time

Work through these on your own before checking the marked answer.

Practice Question 1

What will this code print?

```
correct = 3
total = 0
print(correct / total)
```

- A. 3
- B. ZeroDivisionError ✓**
- C. 0
- D. None

Why: `total` is 0, so `correct / total` raises a `ZeroDivisionError` at runtime — this is an exception, not a syntax error.

Practice Question 2

Which block runs no matter what — whether an exception was raised or not?

- A. `try`
- B. `except`
- C. `else`
- D. `finally` ✓**

Why: `finally` always executes, making it the right place to close files or release resources.

Practice Question 3

Which file mode would you use to add a new line to an existing log file without erasing what's already there?

- A. "r"
- B. "w"
- C. "a" ✓
- D. "r+"

Why: "a" (append) adds to the end of the file; "w" would erase the existing content first.

Practice Question 4

What does this list comprehension produce?

```
print([x**2 for x in range(1, 6)])
```

- A. [1, 4, 9, 16, 25] ✓
- B. [0, 1, 4, 9, 16]
- C. [1, 2, 3, 4, 5]
- D. SyntaxError

*Why: It squares each number from 1 to 5 inclusive: $[x**2 \text{ for } x \text{ in range}(1, 6)] \rightarrow [1, 4, 9, 16, 25]$.*

Practice Question 5

Write a program that prints the reciprocal ($1/x$) of even numbers only. For zero, raise a `ZeroDivisionError`; for odd numbers, raise a `ValueError`. Which exception type should catch BOTH cases in one except clause?

- A. Exception
- B. ArithmeticError
- C. (ZeroDivisionError, ValueError) ✓
- D. BaseException

Why: A tuple of exception types — except (ZeroDivisionError, ValueError): — catches either one in a single clause.



Practice Question 6

What command would you run to install pandas before importing it?

- A. import install pandas
- B. pip install pandas ✓**
- C. python get pandas
- D. pandas.install()

Why: pip is Python's package manager; pip install pandas downloads and installs it so it can then be imported.

Practice Question 7

Which module provides `os.getcwd()` and `os.path.join()`?

- A. math
- B. re
- C. os ✓**
- D. sys

Why: os is the standard library module for interacting with the operating system, including paths and directories.